



矩阵 SVD 分解问题

2026Spring 并行程序设计课程大作业选题

邹博闻

2312251@mail.nankai.edu.cn

2026 年 4 月 3 日

目录

一、 结论	ii
(一) 选题难度	iii
(二) 选题工作量	iii
(三) 给分/助教能够提供的支持	iii
二、 矩阵 SVD 分解	iv
(一) SVD 分解背景 (可以直接跳过)	iv
(二) 为什么要用复杂的算法计算	v
(三) 前置知识	v
(四) Householder 变换 (最好看懂)	vi
(五) 通过 Householder 变换将矩阵上二对角化	vii
(六) Golub-Kahan 算法过程	ix
三、 总结	xii
四、 附录 1: 将矩阵上二对角化的实例	xii
五、 附录 2: 对一个非方阵进行一轮 GKH 算法	xv

一、 结论

与 PCFG 类似，同样先把大家最关注的部分放在前面，在这里简单地介绍一下该选题的优势与劣势。

首先需要明确一点，该实验手册内容较多，这是因为助教为了便于理解算法，插入了很多实际例子和描述，且这些例子都经过助教手算，如果你发现有哪里出错，欢迎私信交流。

选择该选题和框架的劣势：

- 如果你想“看懂”计算过程，可能需要较多时间，至少需要你不要忘记大一上线性代数的内容，同时可能需要额外学习一些内容（主要与数值分析相关），尽管它们的难度并不很高；
- 由于该框架是第一年使用，后续实验的难度难以预期，特别是 GPU 编程实验；
- 使用 AI 进行写作是允许的，包括实验报告的部分内容。实际上目前实现的串行框架也很大程度上使用了 AI。但请注意：**AI 在这里的效果可能并没有你预期的好。**
- 同时，如果助教发现你在对框架不了解的情况下用 **AI 面向答案编程**，你可能面临被判抄袭的风险。实际上这是很有可能的，因为在助教自行实现的过程中，不止一次发现 AI 犯错的情况（尽管目前的 AI 确实足够强大）。

那么相应地，选择该选题和框架也有一些优势：

- 该算法可并行的点非常多。你可以尝试在很多地方优化计算效率（甚至是底层的矩阵乘法和加法），如果你对复杂的并行化感到没有把握，可以多尝试一些较简单的优化。**只要能体现出你的工作和思考，优化的难易程度并不会限制你的得分上限**
- 对这个相对复杂的算法，在前期甚至后期都无法加速是非常正常的。你可以粗略地认为，本框架“**加速比**”**不做严格要求**，只要你保证结果正确性，同时具备分析加速/减速原因的基础能力，得到一个好的分数是没有任何问题的。

- 如果你不想“看懂”这个算法，实际上也是没问题的。目前为了更好地体现并行点，该框架对原始算法做了一定修改，后续每次实验都会给出推荐的并行点。

接下来是对该选题各方面特点的详细介绍。

(一) 选题难度

到目前为止，已完成的任务包括：

- 串行代码的编写，以及正确性的初步测试；
- 找到针对 SIMD、Pthread/OpenMP、MPI 的并行点
- 对 SIMD 和多线程编程进行测试

(二) 选题工作量

尽管前面说算法理解起来有难度，但实际上在有成熟的 AI 帮助下，对着代码，想看懂本题的难度并不高。毕竟本题的绝大部分时间都在处理矩阵乘法。

在以上背景下，本题 SIMD 和多线程任务量不高，且与其他三个框架不同，SIMD 与多线程想实现加速的难度相对低。

个人认为，本框架的 MPI 任务工作量会稍微大。事实上根据去年以及往年情况来看，不管你选哪个框架，MPI 编程总会是一个相对困难的并行任务，本框架也不例外。但此处可以承诺的是，本框架 MPI 环节对并行效率要求会酌情放宽。

(三) 给分/助教能够提供的支持

实际上，2025spring 中三个框架的助教平均分基本一致，所以你可以认为给分好坏与框架无关。对于本选题，同样会控制平均分数与其余选题相对一致。但是助教可以承诺：在批作业时，尽可能给出明确的加分点/扣分点，以及批阅过程中产生的疑问。当然，可能存在时间紧张导致给分理由不够充分的情况，但助教会尽可能避免只给分数，希望同学们能理解，

与其他框架一样，对于选择本选题的同学，如对得分有不同意见，你可以根据助教的点评，针对性地进行反驳 (rebuttal)，并且联系助教。**不论反驳是否合理，反驳后的得分不会低于最初给分。**

如果被判定抄袭，你也可以选择与本人进行 rebuttal。答辩会围绕你写的报告进行，而非助教自由出题。

助教保证能够提供的帮助包括：1. 分析你的代码实现有什么改进方法；2. 额外地为你讲解并行算法思路。

助教会尽力但不保证能提供的帮助：1. 帮你 debug 程序；2. 帮你解决本地的环境配置。特别是 debug，由于并行思路不同，常常是难以提供帮助，而不是不愿帮助。3. 分析你的代码为什么结果不正确。

选择该选题的同学如对作业得分有意见或者需要该选题的相关帮助，请飞书联系邹博闻 (2312251)。

那么接下来，就进入矩阵 SVD 分解的介绍部分，以及本选题所用的算法介绍。

二、 矩阵 SVD 分解

(一) SVD 分解背景 (可以直接跳过)

如果你此前了解矩阵 SVD 分解的作用，可以跳过这一部分。

我们为什么要对矩阵做 SVD 分解？实际上如果你修《人工智能导论》这门课，你很可能会在一次实验里使用它。它是**所有矩阵**都能使用的通用正交分解，**不像特征值分解只局限于方阵**。常见的用途包括以下几个方面：数据降维、去噪压缩、特征提取——实际上人工智能导论其中一次实验就利用了“特征提取”的功能。

简单理解：SVD 把一个复杂矩阵拆成三个结构规整的矩阵相乘。对任意一个 $m \times n$ 的矩阵 A ，它的奇异值分解形式为：

$$A = U \Sigma V^T \quad (1)$$

其中各个矩阵的含义和性质如下：

- U 是一个 $m \times m$ 的正交矩阵，满足 $U^T U = I_m$ 。 U 的列向量称为左奇异向量，它们是由 AA^T 的特征向量构成的；

- Σ 是一个 $m \times n$ 的**对角矩阵**，其对角线上的元素 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ (其中 $r = \min(m, n)$) 称为奇异值，其余位置全为 0。奇异值的大小反映了对应方向上的”能量”或”重要性”；
- V 是一个 $n \times n$ 的**正交矩阵**，满足 $V^T V = I_n$ 。 V 的列向量称为右奇异向量，它们是由 $A^T A$ 的特征向量构成的。

从几何角度理解，SVD 分解将原矩阵 A 表示的线性变换分解为三个步骤：先在 V^T 作用下进行旋转/反射，再在 Σ 作用下沿坐标轴方向进行缩放，最后在 U 作用下再次旋转/反射。

(二) 为什么要用复杂的算法计算

看到这里，你很可能会想：在大一线性代数，就教过计算特征值和特征向量的方法。按这个对 $A^T A$ 和 $A A^T$ 特征值分解，不就能得到 SVD 吗？为什么还要用各种复杂的算法进行 SVD？

实际上这种想法完全没有问题，这也是最基本的 SVD 计算方法，叫做**特征值分解法**。但是，这个方法的计算效率奇臭无比，尤其是对于大规模矩阵来说，计算特征值和特征向量的过程非常耗时。可以想象一下，按照 $\det(A - \lambda I) = 0$ 的方式计算特征值，你将会面对一个最高 n 次的多项式方程，对这个方程的求解是困难且常常不可行的。

因此，在实际应用中，我们通常会采用一些更高效的算法来计算 SVD，例如 **Jacobi 算法**、**Golub-Kahan 算法**等。

这些算法通过迭代的方法，能够更快地收敛到 SVD 的结果，特别适合处理大规模矩阵。本框架采用的就是 **Golub-Kahan 算法**，下面将会对它的原理进行介绍。

(三) 前置知识

Golub-Kahan 算法，简称 GKH 算法，是工业上常用的矩阵 SVD 分解方法之一。但为了介绍这个方法，首先需要再次介绍一些线性代数的知识（其中大部分在大一讲过），如果你愿意，可以看看证明：

- **正交矩阵的定义**：若 $n \times n$ 的方阵 Q 满足 $Q^T Q = Q Q^T = I_n$ ，则称 Q 为正交矩阵。

- **正交矩阵与正交矩阵的积仍是正交矩阵：** 设 P 和 Q 都是 $n \times n$ 的正交矩阵，则 $(PQ)^T(PQ) = Q^T P^T P Q = Q^T (P^T P) Q = Q^T I_n Q = Q^T Q = I_n$ ，因此 PQ 也是正交矩阵。这个性质可以推广到任意有限个正交矩阵的乘积。
- **相似矩阵的特征值相同：** 设 A 和 B 是 $n \times n$ 的方阵，如果存在可逆矩阵 P 使得 $B = P^{-1}AP$ ，则称 A 和 B 相似。对于特征多项式有： $\det(B - \lambda I) = \det(P^{-1}AP - \lambda I) = \det(P^{-1}(A - \lambda I)P) = \det(P^{-1}) \det(A - \lambda I) \det(P) = \det(A - \lambda I)$ （因为 $\det(P^{-1}) \det(P) = \det(P^{-1}P) = \det(I) = 1$ ）。因此 A 和 B 具有相同的特征多项式，从而有相同的特征值。

如果你觉得以上理解起来没什么问题，可以继续往下看：

(四) Householder 变换（最好看懂）

Householder 变换是一种正交变换，它可以将向量的某些分量“反射”为零。对于任意非零向量 $\mathbf{x} \in \mathbb{R}^m$ ，Householder 矩阵定义为：

$$H = I - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}} \quad (2)$$

其中 $\mathbf{v} = \mathbf{x} - \alpha \mathbf{e}_1$ ， $\alpha = \pm \|\mathbf{x}\|_2$ （符号通常取 $-\text{sign}(x_1)$ ）， $\mathbf{e}_1 = (1, 0, \dots, 0)^T$ 是标准基向量。

很显然，这样的理论公式还是太难以理解，下面以一个 3×3 矩阵为例，对其应用 Householder 变换，化为上三角矩阵。

对矩阵

$$A = \begin{pmatrix} 0 & 3 & 1 \\ 0 & 4 & -2 \\ 2 & 1 & 1 \end{pmatrix} \quad (3)$$

第 1 步 $\alpha_1 = (0, 0, 2)^T$ ， $\|\alpha_1\|_2 = 2$ （ $\|\alpha_1\|_2$ 可以理解为向量的模长）。上面说了正负号为 $-\text{sign}(x_1)$ 。这里由于 α_1 的第一个元素为 0，所以 a_1 可正可负，这里取负，最终令 $a_1 = -2$ ；

第 2 步 令 $\omega_1 = \frac{\alpha_1 - a_1 \mathbf{e}_1}{\|\alpha_1 - a_1 \mathbf{e}_1\|_2}$ ，其中 $\mathbf{e}_1 = (1, 0, 0)^T$ ，得 $\omega_1 = \frac{1}{2\sqrt{2}}(1, 0, 1)^T$

第 3 步 令 $H_1 = I - 2\omega_1\omega_1^T$ ，其中 I 是单位矩阵，得

$$H_1 = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix} \quad (4)$$

记 $H'_1 = H_1$ (这里使用 H'_1 是为了和下面的第二轮处理保持一致)。

第 4 步 计算 $H'_1 A$ 。得

$$H'_1 A = \begin{pmatrix} -2 & -1 & -1 \\ 0 & 4 & -2 \\ 0 & -3 & -1 \end{pmatrix} \quad (5)$$

这样就通过变换, 使 A 的第一列中第二行以下的元素全部变为 0 了。

那么如何继续变换呢? 对第二列:

第 1 步 令 $\alpha_2 = (4, -3)^T$, $\|\alpha_2\|_2 = 5$, 且 $-\text{sign}(x_1) = -1$ 记 $a_2 = -5$;

第 2 步 令 $\omega_2 = \frac{\alpha_2 - a_2 e_2}{\|\alpha_2 - a_2 e_2\|_2}$, 其中 $e_2 = (1, 0)^T$, 得 $\omega_2 = \frac{1}{\sqrt{10}}(3, -1)^T$

第 3 步 令 $H_2 = I - 2\omega_2\omega_2^T$, 其中 I 是单位矩阵, 得

$$H_2 = \frac{1}{5} \begin{pmatrix} -4 & 3 \\ 3 & 4 \end{pmatrix} \quad (6)$$

记 $H'_2 = \begin{pmatrix} 1 & 0 \\ 0 & H_2 \end{pmatrix}$ 。请注意, 这里是一个分块矩阵;

第 4 步 计算 $H'_2 H'_1 A$, 得

$$H'_2 H'_1 A = \begin{pmatrix} -2 & -1 & -1 \\ 0 & -5 & 1 \\ 0 & 0 & -2 \end{pmatrix} \quad (7)$$

以上这个例子介绍了将普通矩阵转为上三角阵的方法, 这里助教已经尽可能详细的通过计算实例来介绍了。

(五) 通过 Householder 变换将矩阵上二对角化

对于一个普通的矩阵 A , 其性质往往不够优秀。但前文说过, A 的奇异值完全由 $A^T A$ 和 AA^T 的特征值决定, 所以我们完全可以退而求其次, 找到与 $A^T A/AA^T$ 相似的、且具有”好性质”的矩阵作为代餐。

那么如何找到这样的“好”矩阵呢？高人们给出的一个方法是：通过**上二对角化**进行构造。这部分的理论推导比较复杂，但你只需要知道，通过上二对角化可以将任意 $m \times n$ 的矩阵 A 转化为**上二对角矩阵** (bidiagonal matrix) B ，满足：

$$A = UBV^T \quad (8)$$

其中 U 和 V 都是正交矩阵， B 只有主对角线和次对角线上的元素非零。具体来说， B 的形式如下：

$$B = \begin{pmatrix} \alpha_1 & \beta_1 & 0 & \cdots & 0 \\ 0 & \alpha_2 & \beta_2 & \ddots & \vdots \\ 0 & 0 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{n-1} \\ 0 & \cdots & 0 & 0 & \alpha_n \end{pmatrix} \quad (9)$$

(当 $m > n$ 时，下方还有全零行，本框架不考虑 $m < n$ 的情况)

上二对角化的好处在于：它将原矩阵的非零元素数量从最多 mn 个减少到了最多 $2n - 1$ 个（此处 $m \geq n$ ），大大简化了后续的计算。更重要的是，上二对角矩阵的奇异值与原矩阵 A 的奇异值完全相同。

那么对任意一个 A ，我们都能找到 $A = UBV^T$ 吗？答案是肯定的。

利用 Householder 变换，我们可以逐步将矩阵 A 化为上二对角形式。具体过程如下：对于一个 $m \times n$ 的矩阵 A ($m \geq n$)，上二对角化需要 n 步迭代（实际只需 $n - 1$ 步）。在第 k 步 ($k = 1, 2, \dots, n - 1$)：

1. **左乘 Householder 矩阵 U_k** ：构造 $m \times m$ 的 Householder 矩阵 U_k ，使得 U_k 作用于当前矩阵的第 k 列，将该列第 $k + 1$ 行到第 m 行的元素全部变为零（只保留第 k 行及以上的元素）。
2. **右乘 Householder 矩阵 V_k^T** ：构造 $n \times n$ 的 Householder 矩阵 V_k ，使得 V_k^T 作用于当前矩阵的第 k 行，将该行第 $k + 2$ 列到第 n 列的元素全部变为零（只保留第 k 列和第 $k + 1$ 列的元素）。

经过 $n - 1$ 步迭代后，我们得到：

$$B = U_{n-1}^T \cdots U_2^T U_1^T A V_1 V_2 \cdots V_{n-1} \quad (10)$$

令 $U = U_1 U_2 \cdots U_{n-1}$ ($m \times m$ 正交矩阵), $V = V_1 V_2 \cdots V_{n-1}$ ($n \times n$ 正交矩阵), 则有:

$$A = UBV^T \quad (11)$$

如果你觉得这里对这个算法的描述太过粗糙, 可以先跳至本 pdf 的结尾附录, 有一个实际的例子介绍如何将矩阵上二对角化。

(六) Golub-Kahan 算法过程

经过上面又臭又长的证明, 历经千辛万苦终于可以通过变换由一个 $m \times n$ 矩阵 A (本选题保证 $m \geq n$) 得到一个上二对角矩阵 B 了。在得到 B 矩阵后, 需要做的就是按照流程一步步计算其奇异值。

Golub-Kahan 算法的核心是带 Wilkinson 偏移的隐式 QR 迭代。听起来很复杂, 但实际上流程非常固定:

第 1 步: 计算 Wilkinson 偏移量

对 A 上二对角化得到 B 后, 取得 $B^T B$ 矩阵右下角 2×2 的子矩阵:

$$M = \begin{pmatrix} \alpha_{n-1}^2 + \beta_{n-2}^2 & \alpha_n \beta_{n-1} \\ \alpha_n \beta_{n-1} & \alpha_n^2 \end{pmatrix} \quad (12)$$

计算这个 2×2 矩阵的两个特征值, 选择更接近 α_n^2 的那个作为 Wilkinson 偏移量 μ 。具体计算公式为:

$$\mu = \frac{\text{tr}(M) - \sqrt{\text{tr}(M)^2 - 4\det(M)}}{2} \quad (13)$$

其中 $\text{tr}(M)$ 是矩阵 M 的迹 (对角线元素之和), $\det(M)$ 是 M 的行列式。

为什么要这么做? Wilkinson 偏移可以显著加速 QR 迭代的收敛速度, 特别是当矩阵接近对角化时。

第 2 步: 构造 Givens 旋转

对本选题, 不用在意 Givens 旋转是什么, 你只需要知道它的计算方式。根据上一步确定的偏移量 μ , 构造第一个 Givens 旋转矩阵 G_1 , 使其作用于 $(B^T B - \mu I)$ 的第一列时能够消去第二个元素。Givens 旋转矩阵的形式为:

$$G_1 = \begin{pmatrix} c_1 & s_1 \\ -s_1 & c_1 \end{pmatrix} \quad (14)$$

其中 $c_1 = \cos \theta$, $s_1 = \sin \theta$, 角度 θ 的选择满足:

$$\begin{pmatrix} c_1 & s_1 \\ -s_1 & c_1 \end{pmatrix} \begin{pmatrix} \alpha_1^2 - \mu \\ \alpha_1 \beta_1 \end{pmatrix} = \begin{pmatrix} * \\ 0 \end{pmatrix} \quad (15)$$

解得:

$$c_1 = \frac{\alpha_1^2 - \mu}{\sqrt{(\alpha_1^2 - \mu)^2 + (\alpha_1 \beta_1)^2}}, \quad s_1 = \frac{\alpha_1 \beta_1}{\sqrt{(\alpha_1^2 - \mu)^2 + (\alpha_1 \beta_1)^2}} \quad (16)$$

第 3 步: 将首个 Givens 旋转作用到 B (右乘)

由第 2 步得到的 G_1 应当从右侧作用在 B 上 (作用于前两列):

$$B^{(1)} = BG_1 \quad (17)$$

这里是右乘, 因此本质上是在混合第 1、2 列。对上二对角矩阵而言, 该操作会在位置 (2, 1) 产生一个非零元 (bulge), 从而暂时破坏上二对角结构。

第 4 步: 左乘恢复上二对角结构

接下来构造另一个 Givens 旋转 H_1 ($H_1 y = \begin{pmatrix} * \\ 0 \end{pmatrix}$), 并从左侧作用在 $B^{(1)}$ 上 (作用于前两行):

$$B^{(2)} = H_1 B^{(1)} \quad (18)$$

该步骤用于消去 (2, 1) 处的 bulge。由于左乘会混合前两行, 新的非零元会被“推”到更右侧的 (1, 3), 于是需要进行下一对右乘/左乘旋转。

第 5 步: 追赶 bulge (chase the bulge)

之后重复“右乘一次、左乘一次”的交替过程:

$$B \leftarrow BG_2, \quad B \leftarrow H_2 B, \quad \dots, \quad B \leftarrow BG_{n-1}, \quad B \leftarrow H_{n-1} B \quad (19)$$

每一步都把当前 bulge 向右下角推进, 直到其被“追出”矩阵。

第 6 步: 检查收敛

理论上完成一轮后得到的新矩阵 B' 仍为上二对角矩阵, 且与原 B 具有相同奇异值。接下来的任务是: 扫描主对角线和上次对角线的元素, 判断一下有没有元素收敛到 0。

扫描的结果有三种可能：

情况 1：上次对角线的某元素收敛到 0，如下 $\beta_2 = 0$ ：

$$\begin{pmatrix} \alpha_1 & \beta_1 & 0 & \cdots & 0 \\ 0 & \alpha_2 & \mathbf{0} & \ddots & \vdots \\ 0 & 0 & \alpha_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{n-1} \\ 0 & \cdots & 0 & 0 & \alpha_n \end{pmatrix} \quad (20)$$

那么我们可以将划分为 $\begin{pmatrix} \alpha_1 & \beta_1 \\ 0 & \alpha_2 \end{pmatrix}$ 和 $\begin{pmatrix} \alpha_3 & \beta_3 & 0 & \cdots & 0 \\ 0 & \alpha_4 & \beta_4 & \ddots & \vdots \\ 0 & 0 & \alpha_5 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{n-1} \\ 0 & \cdots & 0 & 0 & \alpha_n \end{pmatrix}$ 两个子矩

阵，分别对其重复 GKH 算法的迭代。

情况 2：主对角线的某元素收敛到 0，如下 $\alpha_3 = 0$ ：

$$\begin{pmatrix} \alpha_1 & \beta_1 & 0 & \cdots & 0 \\ 0 & \alpha_2 & \beta_2 & \ddots & \vdots \\ 0 & 0 & \mathbf{0} & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{n-1} \\ 0 & \cdots & 0 & 0 & \alpha_n \end{pmatrix} \quad (21)$$

此时需要注意的是， $\beta_3 \neq 0$ ，但我们依旧可以利用第五步中“赶气泡”的思路，左乘一系列 Givens 矩阵，将 β_3 元素“赶出矩阵”，这样就回到情况 1 的类

型，可以按情况 1，将矩阵分为 $\begin{pmatrix} \alpha_1 & \beta_1 & 0 \\ 0 & \alpha_2 & \beta_2 \\ 0 & 0 & 0 \end{pmatrix}$ 和 $\begin{pmatrix} \alpha_4 & \beta_4 & 0 & \cdots & 0 \\ 0 & \alpha_5 & \beta_5 & \ddots & \vdots \\ 0 & 0 & \alpha_6 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{n-1} \\ 0 & \cdots & 0 & 0 & \alpha_n \end{pmatrix}$ 两

部分，且前一个矩阵可以通过再右乘一系列 Givens 矩阵的方法，将第三列元素变为 0，最终得到一个 2×2 和一个 $(n-3) \times (n-3)$ 的子矩阵。

情况 3：没有元素收敛，返回第一步进行迭代。

以上就是 Golub-Kahan 算法的全部过程了，在一次次迭代和划分子矩阵后，最终你会获得一系列 1×1 的小子矩阵，它们的元素就构成 B 的奇异值矩阵

Σ ，而把每次 Givens 左乘和右乘的矩阵合并起来，就实现了 $U^T B V = \Sigma$ ，即 $B = U \Sigma V^T$ ，你就成功实现了对 B 矩阵的 SVD 分解。

而 $B = U_{n-1}^T \cdots U_2^T U_1^T A V_1 V_2 \cdots V_{n-1}$ (如果你忘了，可以去看 Householder 变换章节的末尾)，所以 $A = U_1 U_2 \cdots U_{n-1} U \Sigma V^T V_{n-1}^T V_{n-2}^T \cdots V_1^T$ ，而一系列的 U 和 V 都是正交矩阵，这标志着，我们终于完成了对 A 的奇异值分解。

三、 总结

通过以上介绍，相信你也会发现，算法中的各方各面都存在可并行的点：

- 看起来最“高级”的地方：GKH 迭代过程中对**分块矩阵**的处理，这里显然可以通过多线程/多进程进行并行化，让空闲的线程/进程来领取子矩阵，然后迭代直到可以分解为更小的矩阵。但这个过程注定不会是一帆风顺的，因为子矩阵的大小肯定是极其不均衡的，而不均衡就会导致负载不均，“高级”的同时，难度也会很高；
- 不那么“高级”但也有用的一些地方：检测是否有元素收敛到 0 的过程必然可以并行化，且你大概能直观感受到，这里的难度比分块并行要低得多。类似的 for 循环有很多，这里不一一列举，后续实验会给出建议优化的点；
- 看似“低级”但同样很有用的地方：矩阵乘法的优化。目前的矩阵乘法是纯粹的朴素实现，效率显然是不够优秀的，你可以在这里下一些功夫，个人预计结果不会差，因为这个算法充满了各式各样的矩阵乘法。如果你选《算法导论》这门课，你甚至可以尝试优化矩阵乘法的时间复杂度，这也可以记录在你的报告里。

需要特别承诺，选择简单的并行点并不会压低你的得分上限，只要你能在报告里展示你的工作内容和思考，助教不会因优化点不同而区别对待，这点在后续每次实验的任务安排会有实际的说明。

四、 附录 1：将矩阵上二对角化的实例

这里还是以普通矩阵 A 作例子，化为上二对角阵，个人建议可以拿笔算一算。

对矩阵

$$A = \begin{pmatrix} 0 & 3 & 1 \\ 0 & 4 & -2 \\ 2 & 1 & 1 \end{pmatrix} \quad (22)$$

第 1 步 (左乘 H_1 , 消去第一列下方元素)

$\alpha_1 = (0, 0, 2)^T$, $\|\alpha_1\|_2 = 2$, 令 $a_1 = -2$ (第一个元素为 0, 正负均可, 这里取负);

令 $\omega_1 = \frac{\alpha_1 - a_1 e_1}{\|\alpha_1 - a_1 e_1\|_2}$, 其中 $e_1 = (1, 0, 0)^T$, 得 $\omega_1 = \frac{1}{2\sqrt{2}}(1, 0, 1)^T$;
令 $H_1 = I - 2\omega_1\omega_1^T$, 得

$$H_1 = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix} \quad (23)$$

记 $B = H_1 A$, 计算得

$$B = \begin{pmatrix} -2 & -1 & -1 \\ 0 & 4 & -2 \\ 0 & -3 & -1 \end{pmatrix} \quad (24)$$

此时第一列下方已清零。

第 2 步 (右乘 Q_1 , 消去第一行第三列元素)

取 B 的第一行后两个元素 $\beta_1 = (-1, -1)^T$ (视为列向量), $\|\beta_1\|_1 = \sqrt{2}$, 令 $b_1 = \sqrt{2}$ (与 β_1 的第一个元素 -1 异号);

令 $\omega_2 = \frac{\beta_1 - b_1 e_1}{\|\beta_1 - b_1 e_1\|}$;

则 2×2 Householder 矩阵为 $\tilde{Q}_1 = I - 2\omega_2\omega_2^T$, 经计算得

$$\tilde{Q}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 & -1 \\ -1 & 1 \end{pmatrix} \quad (25)$$

扩展为 3×3 矩阵 (作用于第 2、3 列):

$$Q_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ 0 & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \quad (26)$$

右乘 B 得 $C = BQ_1$:

$$C = \begin{pmatrix} -2 & \sqrt{2} & 0 \\ 0 & -\sqrt{2} & -3\sqrt{2} \\ 0 & 2\sqrt{2} & \sqrt{2} \end{pmatrix} \quad (27)$$

此时第一行第三列也已清零。

第 3 步 (左乘 H_2 , 消去第二列下方元素)

取 C 的第二列第 2、3 行元素 $\alpha_2 = (-\sqrt{2}, 2\sqrt{2})^T$, $\|\alpha_2\|_2 = \sqrt{10}$, 令 $a_2 = +\sqrt{10}$ (因为 $-\text{sign}(-\sqrt{2}) = -(-1) = +1$, 取正);

令 $\omega_3 = \frac{\alpha_2 - a_2 e_1}{\|\alpha_2 - a_2 e_1\|_2}$, 其中 $e_1 = (1, 0)^T$, 得

$$\alpha_2 - a_2 e_1 = \begin{pmatrix} -\sqrt{2} - \sqrt{10} \\ 2\sqrt{2} \end{pmatrix},$$

$$\|\alpha_2 - a_2 e_1\|_2 = \sqrt{(\sqrt{2} + \sqrt{10})^2 + (2\sqrt{2})^2} = \sqrt{20 + 4\sqrt{5}} = 2\sqrt{5 + \sqrt{5}}$$

$$\omega_3 = \frac{1}{2\sqrt{5 + \sqrt{5}}} \begin{pmatrix} -\sqrt{2} - \sqrt{10} \\ 2\sqrt{2} \end{pmatrix}$$

构造 2×2 Householder 矩阵 $\tilde{H}_2 = I - 2\omega_3\omega_3^T$, 经化简得

$$\tilde{H}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 & 2 \\ 2 & 1 \end{pmatrix} \quad (28)$$

扩展为 3×3 矩阵 (作用于第 2、3 行):

$$H_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -\frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \\ 0 & \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \end{pmatrix} \quad (29)$$

左乘 C 得 $D = H_2 C$:

$$D = \begin{pmatrix} -2 & \sqrt{2} & 0 \\ 0 & \sqrt{10} & \sqrt{10} \\ 0 & 0 & -\sqrt{10} \end{pmatrix}$$

最终结果矩阵 D 已经是上二对角形式（只有主对角线和上一次对角线有非零元）：

$$\begin{pmatrix} -2 & \sqrt{2} & 0 \\ 0 & \sqrt{10} & \sqrt{10} \\ 0 & 0 & -\sqrt{10} \end{pmatrix} \quad (30)$$

实际上对更大规模的方阵/非方阵进行上二对角化的过程是完全一样的，这里你可以自己写一个矩阵算一算。

五、 附录 2： 对一个非方阵进行一轮 GKH 算法

我们按照前文所讲的流程，逐步执行。

不妨假设已经得到了上二对角阵 $B = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$

第一步 取 $B^T B$ 右下角 2×2 的子矩阵：

$$B^T B = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{pmatrix} \quad (31)$$

其右下角矩阵的特征值为 3 和 1，我们选取相距右下角更近的值作为 Wilkinson 偏移，此处选 $\mu = 3$ 。

第二步 构造一个 Givens 变换 $G_1 = \begin{pmatrix} c & s \\ -s & c \end{pmatrix}$ ，使得

$$G_1 \begin{pmatrix} \alpha_1^2 - \mu \\ \alpha_1 \beta_1 \end{pmatrix} = \begin{pmatrix} * \\ 0 \end{pmatrix} \quad (32)$$

不难解得 $c = -\frac{2}{\sqrt{5}}, s = \frac{1}{\sqrt{5}}$

第三步 将 G_1 扩展到 3×3 矩阵，然后右乘在 B 上：

$$BG_1 = B \begin{pmatrix} c & s & 0 \\ -s & c & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} -\frac{3}{\sqrt{5}} & -\frac{1}{\sqrt{5}} & 0 \\ -\frac{1}{\sqrt{5}} & -\frac{2}{\sqrt{5}} & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \quad (33)$$

你会发现，刚才的相乘操作会在 (2,1) 处引入一个“突起” (bulge)。

第四步 接下来我们的目的是，再构造一个 Givens 矩阵 G_2 ，使 G_2B 结果中 (2,1) 处的气泡变为 0，因此我们取 (1,1) 和 (2,1) 构成列向量 $\begin{pmatrix} -\frac{3}{\sqrt{5}} \\ -\frac{1}{\sqrt{5}} \end{pmatrix}$ ，然后计算 Givens 矩阵：

$$G_2 \begin{pmatrix} -\frac{3}{\sqrt{5}} \\ -\frac{1}{\sqrt{5}} \end{pmatrix} = \begin{pmatrix} * \\ 0 \end{pmatrix} \quad (34)$$

不难解得 $c = \frac{3}{\sqrt{10}}, s = \frac{1}{\sqrt{10}}$ ，然后扩展 G_2 到 4*4 矩阵。并左乘在第三步的结果上：

$$G_2BG_1 = \begin{pmatrix} c & s & 0 & 0 \\ -s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} BG_1 = \begin{pmatrix} -\sqrt{2} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{10}} \\ 0 & -\frac{1}{\sqrt{2}} & \frac{3}{\sqrt{10}} \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \quad (35)$$

不难发现，经过这样的变换后，(2,1) 位置的 bulge 被“赶到了” (1,3) 位置，这与我们前面的理论过程是一致的。

第五步 第四步中我们把 bulge 向右“赶”，现在我们需要将它向下“赶”。因此我们选取 (1,2) 和 (1,3) 元素构成行向量，并计算 Givens 矩阵：

$$\begin{pmatrix} -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{10}} \end{pmatrix} G_3 = \begin{pmatrix} * & 0 \end{pmatrix} \quad (36)$$

这里的计算略麻烦，但计算机显然是可以把它做好的。这里我们可以放心地假设，此处可以求出新的 c 和 s ，然后就可以把 G_3 扩展到一个 3*3 矩阵：

$$G_2BG_1G_3 = G_2BG_1 \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & s \\ 0 & -s & c \end{pmatrix} = \begin{pmatrix} * & * & 0 \\ 0 & * & * \\ 0 & + & * \\ 0 & 0 & 0 \end{pmatrix} \quad (37)$$

你可以思考一下 G_i 扩展时, c 和 s 应该放到什么位置。实际上, 就第五步来说, 因为我们的目的是对 B 的第二列和第三列做线性组合以消去 (1,3), 根据式 (36) 的原理, 我们自然要把 c 和 s 放到第二列和第三列的位置。

第六步 经过第五步右乘的操作, 我们成功把 bulge 从 (1,3) “赶到了” (3,2), 所以接下来我们要构造一个 Givens 矩阵, 使得

$$G_4 \begin{pmatrix} b_{22} \\ b_{32} \end{pmatrix} = \begin{pmatrix} * \\ 0 \end{pmatrix} \quad (38)$$

这显然也是能做到的。求出 G_4 后, 将其扩展至 4×4 , 然后作用到 B 上:

$$G_4 G_2 B G_1 G_3 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & c & s & 0 \\ 0 & -s & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} G_2 B G_1 G_3 = \begin{pmatrix} * & * & 0 \\ 0 & * & * \\ 0 & 0 & * \\ 0 & 0 & 0 \end{pmatrix} \quad (39)$$

这样经过右乘左乘交替, 我们让 B 恢复上二对角阵, 此时扫描主对角线及上次对角线有无元素收敛至 0 即可。